



Université Hassan II de Casablanca
Faculté des Sciences & Techniques Mohammed VI

Licence IRM — Rapport : Environnements Python et reproductibilité

ENVIRONNEMENTS PYTHON

Installation, configuration et reproductibilité

Python • VS Code • venv • pip • Conda • Anaconda

Guide pratique et méthodologique
pour le développement Python reproductible

Enseignant : Prof. Abdellah ADIB

Année universitaire : 2026–2027

Public : Informatique, Data Science, Intelligence Artificielle

Table des matières

Préambule	iv
Objectifs pédagogiques	v
1 Python, interpréteur et environnement d'exécution	2
1.1 Python comme langage et comme implémentation	2
1.2 Que se passe-t-il avec <code>python programme.py</code> ?	2
1.3 L'environnement d'exécution	3
2 Le PATH et la sélection de l'exécutable	4
2.1 Le rôle du PATH	4
2.2 Commandes de diagnostic	4
3 Installation sous Windows, macOS et Linux	7
3.1 Windows	7
3.2 macOS	7
3.3 Linux	8
3.4 Version recommandée ou version imposée?	8
4 pip et l'installation des paquets	9
4.1 Qu'est-ce que pip?	9
4.2 Commandes essentielles	9
5 Visual Studio Code pour Python	11
5.1 Le rôle de VS Code	11
5.2 Installation et extensions	11
5.3 Terminal intégré et interpréteur VS Code	12
6 Comprendre les environnements virtuels	14
6.1 Pourquoi isoler les projets?	14
6.2 Ce que fait réellement <code>venv</code>	14
7 Créer et utiliser un <code>venv</code>	15
7.1 Structure du projet fil rouge	15
7.2 Création	15
7.3 Activation	16

7.4	Vérification	16
7.5	Installation des bibliothèques	16
7.6	Désactivation et suppression	17
7.7	Le fichier .gitignore	17
8	Comprendre et figer les dépendances	19
8.1	Dépendances directes et transitives	19
8.2	Figer avec pip freeze	19
8.3	Reconstruction sur une autre machine	20
9	Reproductibilité : ce que l'on peut réellement garantir	21
9.1	Plusieurs niveaux de reproductibilité	21
9.1.1	Reproductibilité du code	21
9.1.2	Reproductibilité logicielle	21
9.1.3	Reproductibilité des données	21
9.1.4	Reproductibilité expérimentale	21
9.2	Limites d'un requirements.txt	22
9.3	Écosystème moderne	22
10	L'écosystème Conda	24
10.1	Conda, Miniconda et Anaconda Distribution	24
10.2	Pourquoi Conda est-il utilisé?	24
11	Créer, utiliser et exporter un environnement Conda	25
11.1	Création	25
11.2	Plusieurs versions de Python	25
11.3	Conda dans VS Code	25
11.4	Conda et pip ensemble	26
11.5	Export et reconstruction	26
12	venv + pip ou Conda?	27
12.1	Arbre de décision	27
13	Cas pratiques de configuration	29
13.1	Cas A — Débutant avec un seul projet	29
13.2	Cas B — Deux projets incompatibles	30
13.3	Cas C — Un enseignant fournit un projet	30
13.4	Cas D — Changement d'ordinateur	30
13.5	Cas E — ModuleNotFoundError	30

13.6 Cas F — pip installe dans le mauvais Python	31
14 Tableau de dépannage	32
15 Checklist universelle de diagnostic	34
16 Construire un projet Python reproductible : la méthode de référence	36
16.1 Variante A — Python + venv + pip	36
16.2 Variante B — Conda	37
17 Bonnes pratiques professionnelles	38
18 Fiche mémo — Environnements Python	39
Conclusion	41
Références et ressources officielles	42

Licence IRM

Préambule

Un projet Python ne se résume pas à un fichier `.py`. Pour qu'un programme soit réellement exploitable par une autre personne, sur une autre machine et à un autre moment, il faut maîtriser l'ensemble de son **environnement d'exécution** : version de Python, interpréteur utilisé, paquets installés, versions de ces paquets, variables d'environnement, système d'exploitation et, dans certains projets scientifiques, bibliothèques natives ou accélération GPU.

Ce rapport propose une méthode progressive : partir d'une machine vierge, installer Python, comprendre le rôle du `PATH`, configurer Visual Studio Code, isoler les projets avec `venv`, gérer les paquets avec `pip`, figer les dépendances, reconstruire un environnement ailleurs, puis étudier l'approche alternative proposée par Conda, Miniconda et Anaconda Distribution.

À retenir

Un environnement Python professionnel est un ensemble contrôlé composé d'une **version de Python**, d'un **ensemble de dépendances**, d'une **configuration d'exécution** et d'outils permettant de **reconstruire cet ensemble de manière fiable**.

Objectifs pédagogiques

À l'issue de ce document, l'étudiant doit être capable de :

- ◆ distinguer langage, interpréteur, éditeur, terminal, environnement d'exécution, environnement virtuel et gestionnaire de paquets ;
- ◆ installer Python sous Windows, macOS et Linux et vérifier l'exécutable réellement utilisé ;
- ◆ expliquer le rôle du `PATH`, de `python`, `python3`, `py` et `python -m pip` ;
- ◆ configurer VS Code et sélectionner explicitement le bon interpréteur ;
- ◆ créer, activer, utiliser, diagnostiquer et supprimer un environnement `venv` ;
- ◆ installer, inspecter et figer des dépendances ;
- ◆ reconstruire un environnement sur une autre machine ;
- ◆ comprendre les limites d'un simple `requirements.txt` ;
- ◆ utiliser Conda comme alternative et distinguer Conda, Miniconda et Anaconda Distribution ;
- ◆ appliquer une procédure de diagnostic systématique en cas de problème.

PARTIE I

Comprendre l'écosystème Python

Licence IRM

Python, interpréteur et environnement d'exécution

1.1 Python comme langage et comme implémentation

Python est d'abord un **langage de programmation** : il définit une syntaxe, des règles sémantiques et un modèle d'exécution. Le programme que l'on installe sur une machine est une **implémentation** de ce langage. L'implémentation la plus courante est **CPython**, distribuée par la Python Software Foundation.

Un fichier `programme.py` contient du code source. Il n'est pas, à lui seul, « Python installé ». Il doit être lu et exécuté par un interpréteur compatible.

Pourquoi cette distinction est-elle importante ?

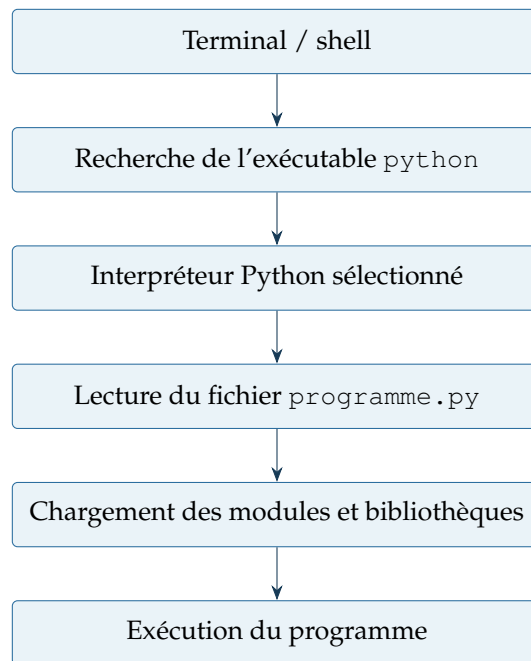
Deux machines peuvent contenir exactement le même fichier `.py` mais exécuter des versions différentes de Python ou disposer de bibliothèques différentes. Le code source est identique, mais l'environnement d'exécution ne l'est pas.

1.2 Que se passe-t-il avec `python programme.py` ?

Commande

```
python programme.py
```

Conceptuellement, le système suit la chaîne suivante :



1.3 L'environnement d'exécution

L'environnement d'exécution comprend tout ce qui influence concrètement l'exécution d'un programme :

- ◆ la version de Python;
- ◆ le système d'exploitation;
- ◆ l'architecture matérielle;
- ◆ les paquets Python et leurs versions;
- ◆ certaines bibliothèques système;
- ◆ les variables d'environnement;
- ◆ éventuellement CUDA, cuDNN, pilotes GPU ou bibliothèques natives.

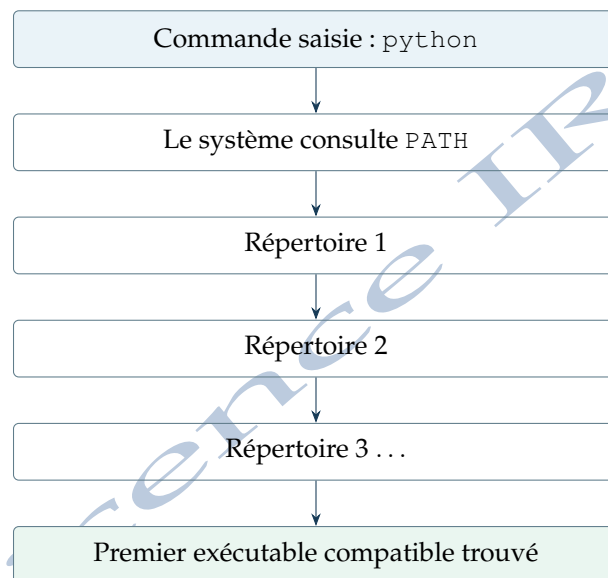
À retenir

Même code source $\neq$ même comportement garanti
si les environnements d'exécution diffèrent.

Le PATH et la sélection de l'exécutable

2.1 Le rôle du PATH

Lorsque l'utilisateur tape simplement `python`, le terminal doit trouver un programme nommé `python`. Il consulte pour cela une liste de répertoires enregistrée dans une variable d'environnement appelée `PATH`.



L'ordre du `PATH` est donc important : deux installations de Python peuvent coexister, mais la première trouvée est généralement celle qui sera lancée par le shell pour la commande considérée.

2.2 Commandes de diagnostic

WINDOWS

Terminal

```
where python
where pip
py --list
```

macOS / LINUX

Terminal

```
which python
which python3
which pip
which pip3
```

Sur toutes les plateformes, Python lui-même peut révéler son chemin :

Terminal

```
python -c "import sys; print(sys.executable)"
python -c "import sys; print(sys.version)"
```

Diagnostic

En cas de doute, ne commencez pas par réinstaller des paquets. Commencez par identifier **quel Python s'exécute réellement**. Le chemin contenu dans `sys.executable` est souvent l'information la plus utile du diagnostic.

Erreur fréquente

Un étudiant exécute `pip install pandas`, puis obtient `ModuleNotFoundError: No module named 'pandas'`. Une cause classique est que `pip` appartient à un autre Python que celui qui exécute le programme.

PARTIE II

Installer et configurer Python

Licence IRM

Installation sous Windows, macOS et Linux

3.1 Windows

L'installation standard s'effectue à partir du site officiel de Python. Pendant l'installation, l'ajout de Python au `PATH` simplifie l'utilisation depuis le terminal. Sous Windows, le lanceur `py` permet aussi de cibler une version installée.

Vérifications

```
python --version
py --version
where python
```

Si plusieurs versions sont disponibles :

Terminal

```
py -3
py -3.12
py -3.13
```

Attention

Une machine Windows peut contenir plusieurs Python : installation officielle, Microsoft Store, environnement Conda, ancien Python conservé pour un projet, etc. La commande `where python` permet de voir plusieurs candidats éventuels.

3.2 macOS

Sur macOS, on utilisera souvent `python3`. Il est déconseillé de modifier aveuglément un Python fourni ou utilisé par le système. Les méthodes fréquentes sont l'installateur officiel ou Homebrew.

Terminal

```
python3 --version
which python3
```

3.3 Linux

De nombreuses distributions disposent déjà de Python. Sur Debian/Ubuntu et dérivées, les paquets nécessaires peuvent être installés ainsi :

Terminal

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv
```

Attention

Les commandes varient selon la distribution. Sur Linux, évitez d'utiliser `sudo pip install ...` dans le Python système pour des projets ordinaires : créez plutôt un environnement virtuel.

3.4 Version recommandée ou version imposée ?

Le site officiel Python indique actuellement Python 3.14 comme branche de fonctionnalité la plus récente. Dans un cadre pédagogique ou scientifique, la bonne pratique n'est pas de « prendre toujours la plus récente » sans réflexion, mais de choisir une version compatible avec les bibliothèques et outils du projet, puis de **la documenter explicitement**.

Bonne pratique

Dans les exemples de ce rapport, des versions comme 3.12 ou 3.13 sont parfois utilisées pour illustrer les commandes. Elles constituent des exemples : la version de référence d'un projet doit être fixée dans son README ou dans son fichier de configuration.

pip et l'installation des paquets

4.1 Qu'est-ce que pip ?

pip est le gestionnaire de paquets Python recommandé pour installer des distributions de projets publiées, notamment depuis Python Package Index (PyPI).

La commande :

Terminal

```
pip install numpy
```

peut être pratique, mais la forme suivante réduit les ambiguïtés :

Terminal

```
python -m pip install numpy
```

Pourquoi ?

`python -m pip` signifie : « demander à **ce Python précis** d'exécuter son module `pip` ». On lie ainsi explicitement l'installateur au Python utilisé par la commande `python`.

4.2 Commandes essentielles

Terminal

```
python -m pip --version
python -m pip list
python -m pip show numpy
python -m pip install numpy
python -m pip install --upgrade numpy
python -m pip uninstall numpy
```

PARTIE III

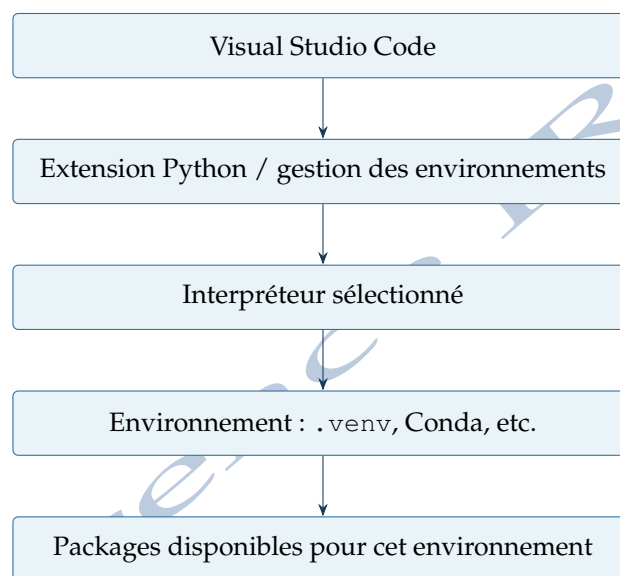
VS Code et environnement d'exécution

Licence IRM

Visual Studio Code pour Python

5.1 Le rôle de VS Code

Visual Studio Code est un **éditeur de code extensible**. Il n'est ni Python, ni un environnement virtuel. L'extension Python ajoute des fonctions de découverte d'interpréteurs, d'exécution, de débogage, de tests et d'intégration avec les environnements.



5.2 Installation et extensions

Installer VS Code, puis l'extension Python officielle de Microsoft. Selon les besoins, ajouter Pylance et l'extension Jupyter.

Dans la palette de commandes :

VS Code

```
Ctrl + Shift + P  
Python: Select Interpreter
```

La sélection de l'interpréteur doit ensuite être vérifiée dans le programme :

```
import sys  
  
print(sys.executable)  
print(sys.version)
```

5.3 Terminal intégré et interpréteur VS Code

Le terminal intégré est un shell. Il peut être automatiquement activé sur l'environnement sélectionné, mais il ne faut pas supposer que l'interpréteur utilisé par une action de VS Code et celui utilisé dans un ancien terminal ouvert sont toujours identiques.

Terminal et VS Code utilisent deux Python différents

Supposons que le terminal affiche Python 3.13 global alors que VS Code a sélectionné un Python 3.12 situé dans `.venv`. Une installation faite dans le terminal global ne sera pas visible dans le `.venv`. Inversement, un paquet installé dans le `.venv` peut sembler « absent » lorsqu'on lance le programme avec le Python global.

Méthode de vérification systématique

- ◆ dans le terminal, exécuter `python -c "import sys; print(sys.executable)";`
- ◆ dans le programme VS Code, afficher `sys.executable`;
- ◆ comparer les deux chemins;
- ◆ vérifier ensuite `python -m pip -version`.

PARTIE IV

venv, pip et isolation des projets

Licence IRM

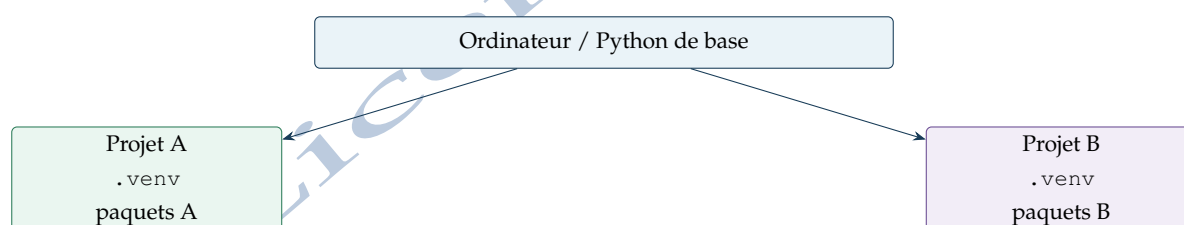
Comprendre les environnements virtuels

6.1 Pourquoi isoler les projets ?

Imaginons deux projets :

	Projet A	Projet B
NumPy	version 1.x	version 2.x
Autres contraintes	pandas 2.x	bibliothèque exigeant une combinaison spécifique

Installer tout globalement finit par créer des conflits. Un environnement virtuel fournit à chaque projet son propre ensemble de paquets.



6.2 Ce que fait réellement venv

Le module standard `venv` crée un environnement léger construit au-dessus d'une installation Python existante appelée Python de base. L'environnement possède notamment son propre répertoire de paquets `site-packages` et, par défaut, il est isolé des paquets du Python de base.

Sous le capot

Dire qu'un `venv` « copie Python » est trop simplificateur. Selon la plateforme et l'installation, l'environnement peut contenir des copies ou des liens vers certains exécutables et fichiers du Python de base. L'idée essentielle est qu'il dispose d'un **contexte d'exécution isolé** et de son propre espace de paquets.

Créer et utiliser un venv

7.1 Structure du projet fil rouge

FIL ROUGE — analyse_donnees

Nous utiliserons le projet `analyse_donnees` tout au long du rapport.

```
analyse_donnees/  
  .venv/  
  src/  
  main.py  
  data/  
  notebooks/  
  tests/  
  requirements.txt  
  .gitignore  
  README.md
```

Le dossier `.venv` contient l'environnement local ; `src` le code ; `tests` les tests ; `requirements.txt` une description figée des dépendances ; `README.md` la procédure de mise en route.

7.2 Création

WINDOWS

Terminal

```
py -m venv .venv
```

macOS / LINUX

Terminal

```
python3 -m venv .venv
```

La commande peut être lue mot à mot :

- ♦ `python` ou `py` : interpréteur de départ ;
- ♦ `-m venv` : exécuter le module standard `venv` ;
- ♦ `.venv` : nom du répertoire cible.

7.3 Activation

WINDOWS POWERSHELL

Terminal

```
.\.venv\Scripts\Activate.ps1
```

WINDOWS CMD

Terminal

```
.venv\Scripts\activate.bat
```

macOS / LINUX

Terminal

```
source .venv/bin/activate
```

Le préfixe (`.venv`) visible dans le terminal n'est qu'un indicateur. L'effet essentiel de l'activation est la modification temporaire de l'environnement du shell, notamment l'ordre du `PATH`, afin que les commandes `python` et `pip` pointent en priorité vers l'environnement.

À retenir

Activer un venv ne « démarre » pas un serveur et ne crée pas une nouvelle session Python permanente. L'activation modifie surtout le contexte du shell courant.

7.4 Vérification

Terminal

```
python -c "import sys; print(sys.executable)"  
python -m pip --version
```

Le chemin doit contenir le répertoire du projet et `.venv`.

7.5 Installation des bibliothèques

Terminal

```
python -m pip install --upgrade pip  
python -m pip install numpy pandas matplotlib scikit-learn  
python -m pip list
```

```
import numpy as np
import pandas as pd
import matplotlib
import sklearn

print("Environnement operationnel")
```

7.6 Désactivation et suppression

Terminal

```
deactivate
```

Pour supprimer le venv, désactivez-le puis supprimez le dossier `.venv`. Il n'est généralement pas nécessaire de « désinstaller » les paquets un par un.

Bonne pratique

Le venv est jetable. Ce qui doit être conservé, c'est le code source et la **description permettant de reconstruire l'environnement**.

7.7 Le fichier `.gitignore`

```
.venv/
__pycache__/
*.pyc
# .vscode/ # à exclure ou à versionner selon la politique du projet
```

Le dossier `.vscode` mérite une nuance : certaines configurations communes peuvent être versionnées dans un projet collaboratif, tandis que des préférences strictement personnelles ne devraient pas l'être.

PARTIE V

Dépendances et reproductibilité

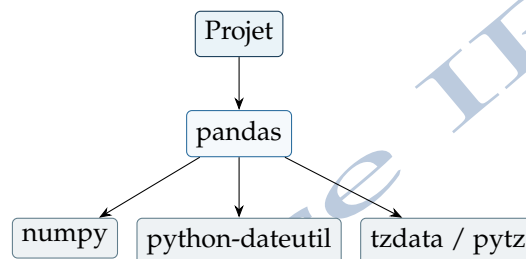
Licence IRM

Comprendre et figer les dépendances

8.1 Dépendances directes et transitives

Installer pandas peut entraîner l'installation d'autres paquets nécessaires à son fonctionnement. On distingue :

- ◆ les **dépendances directes**, choisies explicitement par le projet ;
- ◆ les **dépendances transitives**, nécessaires aux dépendances directes.



8.2 Figer avec pip freeze

Terminal

```
python -m pip freeze
python -m pip freeze > requirements.txt
```

Exemple :

```
numpy==2.2.1
pandas==2.2.3
matplotlib==3.10.0
scikit-learn==1.6.1
```

L'opérateur == demande une version exacte. Cette précision augmente la reproductibilité logicielle, mais ne la rend pas absolue.

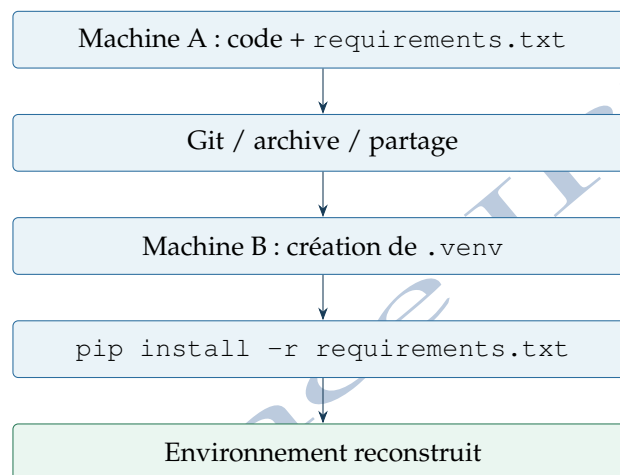
8.3 Reconstruction sur une autre machine

FIL ROUGE — analyse_donnees

Machine A : le projet est terminé et partagé via Git ou archive. Machine B : l'étudiant veut reproduire l'environnement.

Machine B

```
python -m venv .venv
# activer l'environnement
python -m pip install -r requirements.txt
```



Reproductibilité : ce que l'on peut réellement garantir

9.1 Plusieurs niveaux de reproductibilité

9.1.1 Reproductibilité du code

Le même code source doit être disponible, idéalement versionné.

9.1.2 Reproductibilité logicielle

La version de Python et les versions des dépendances doivent être connues et suffisamment contrôlées.

9.1.3 Reproductibilité des données

Le même jeu de données, ou une version précisément identifiée, doit être disponible.

9.1.4 Reproductibilité expérimentale

Les paramètres, les seeds, le matériel et certaines options de calcul doivent être maîtrisés.

```
import random
import numpy as np

random.seed(42)
np.random.seed(42)
```

Attention

Fixer une graine aléatoire ne garantit pas toujours une reproductibilité bit-à-bit, notamment avec certains algorithmes parallèles ou certaines opérations GPU. Les bibliothèques de deep learning disposent de réglages supplémentaires pour limiter les sources de non-déterminisme.

9.2 Limites d'un requirements.txt

Un `requirements.txt` ne capture pas automatiquement :

- ◆ le système d'exploitation ;
- ◆ l'architecture CPU ;
- ◆ la version exacte de Python, sauf si elle est documentée ailleurs ;
- ◆ des bibliothèques natives du système ;
- ◆ les pilotes GPU, CUDA ou cuDNN ;
- ◆ certaines variables d'environnement.

À retenir

Même `requirements.txt` + Python différent + OS différent = résultat potentiellement différent.

9.3 Écosystème moderne

Pour des projets plus avancés, on rencontre aujourd'hui `pyproject.toml`, les fichiers de verrouillage, Poetry, uv, pip-tools et les conteneurs Docker. Ces outils cherchent notamment à mieux exprimer les dépendances directes, accélérer les installations ou améliorer la reproductibilité et le packaging.

Pourquoi d'autres outils existent-ils si `venv` + `pip` fonctionne ?

Parce que `venv` + `pip` résout très bien l'isolation et l'installation de base, mais ne couvre pas à lui seul tous les besoins : résolution reproductible multi-plateforme, gestion de projet, verrouillage précis, publication de paquets, synchronisation rapide ou encapsulation complète du système.

PARTIE VI

Conda, Miniconda et Anaconda

Licence IRM

L'écosystème Conda

10.1 Conda, Miniconda et Anaconda Distribution

Conda	:	Gestionnaire de paquets et d'environnements, capable de gérer Python mais aussi certaines dépendances non Python.
Miniconda	:	Installation minimale fournissant Conda, Python et un petit ensemble de composants essentiels.
Anaconda Distribution	:	Distribution plus complète comprenant Conda, Python, Anaconda Navigator et un grand ensemble de paquets orientés data science et IA.

À retenir

Anaconda n'est pas Conda. Conda est l'outil de gestion; Miniconda et Anaconda Distribution sont deux manières de l'installer avec des contenus initiaux différents.

10.2 Pourquoi Conda est-il utilisé?

Conda est populaire dans les contextes où les projets combinent paquets Python et composants binaires ou natifs : calcul scientifique, data science, machine learning, géosciences, bioinformatique, traitement d'image, etc.

Créer, utiliser et exporter un environnement Conda

11.1 Création

Terminal

```
conda create -n projet_ml python=3.12
conda activate projet_ml
```

Cette commande crée un environnement nommé `projet_ml` avec une version de Python demandée explicitement.

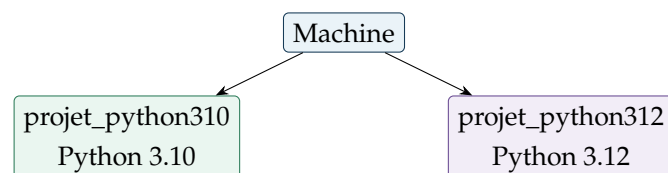
Terminal

```
conda install numpy pandas matplotlib scikit-learn
python --version
conda list
```

11.2 Plusieurs versions de Python

Terminal

```
conda create -n projet_python310 python=3.10
conda create -n projet_python312 python=3.12
```



11.3 Conda dans VS Code

Après création de l'environnement, ouvrir la palette de commandes et sélectionner Python : Select Interpreter. Choisir l'environnement Conda voulu puis vérifier :

```
import sys
```

```
print(sys.executable)
```

11.4 Conda et pip ensemble

Les deux peuvent coexister, mais il faut limiter les mélanges non contrôlés. Une stratégie prudente consiste à installer avec Conda ce qui est disponible et cohérent dans l'écosystème choisi, puis à utiliser `pip` seulement pour les paquets manquants, en documentant le résultat.

Bonne pratique

Dans un environnement Conda, installez d'abord les dépendances Conda majeures. Ajoutez ensuite les paquets `pip` nécessaires. Évitez d'alterner sans méthode entre les deux gestionnaires, car cela peut compliquer la résolution de dépendances.

11.5 Export et reconstruction

La commande historique suivante reste prise en charge :

Terminal

```
conda env export > environment.yml
```

Les versions récentes de Conda proposent aussi `conda export`, avec plusieurs formats de sortie. Pour un support pédagogique interopérable, `environment.yml` reste très courant.

```
name: projet_ml
dependencies:
  - python=3.12
  - numpy
  - pandas
  - scikit-learn
```

Reconstruction :

Terminal

```
conda env create -f environment.yml
```

Suppression :

Terminal

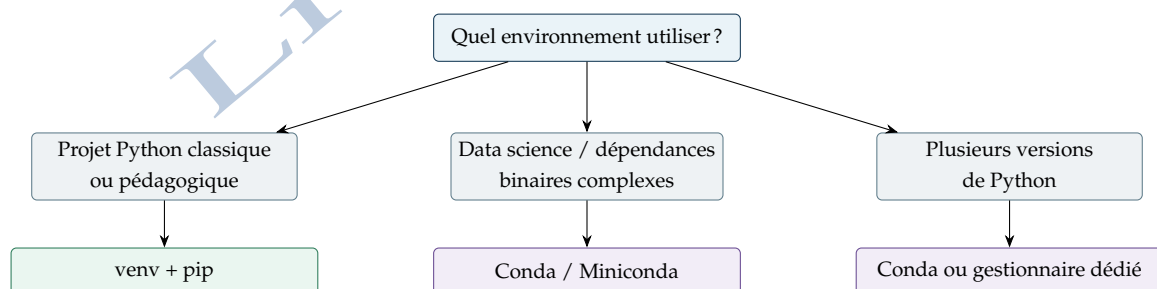
```
conda deactivate
conda env remove -n projet_ml
```


venv + pip ou Conda ?

TABLE 12.1 – Comparaison pédagogique des deux approches

Critère	venv + pip	Conda
Inclus avec Python	venv oui; pip généralement fourni	Non, installation séparée
Gestion d'environnements	Oui	Oui
Paquets Python	Via pip	Via conda, parfois complété par pip
Dépendances non Python	Plus limitée	Plus étendue
Choix de version Python	Dépend des interpréteurs présents	Très simple à exprimer dans l'environnement
Légèreté	Très bonne	Plus lourd
Projet Python classique	Excellent choix	Bon choix
Data science scientifique	Bon	Très courant et pratique
Fichier de partage	requirements.txt / config projet	environment.yml ou formats conda export

12.1 Arbre de décision



PARTIE VII

Cas pratiques, diagnostic et dépannage

Licence IRM

Cas pratiques de configuration

13.1 Cas A — Débutant avec un seul projet

Étape 1 — Installer Python et VS Code

Installer Python depuis la source officielle, vérifier la version, puis installer VS Code et son extension Python.

Étape 2 — Créer le projet

Terminal

```
mkdir analyse_donnees
cd analyse_donnees
python -m venv .venv
```

Étape 3 — Activer et vérifier

Activer selon le système, puis :

Terminal

```
python -c "import sys; print(sys.executable)"
```

Étape 4 — Installer les dépendances

Terminal

```
python -m pip install numpy pandas matplotlib scikit-learn
python -m pip freeze > requirements.txt
```

Étape 5 — Configurer VS Code

Sélectionner l'interpréteur dans `.venv` et vérifier avec `sys.executable`.

13.2 Cas B — Deux projets incompatibles

Créer un `.venv` dans chaque projet. Ne jamais réutiliser artificiellement le même environnement uniquement pour économiser de l'espace : l'isolation fait partie du contrat du projet.

13.3 Cas C — Un enseignant fournit un projet

Le paquet pédagogique partagé doit typiquement contenir :

```
README.md
requirements.txt
src/
data/
tests/
.gitignore
```

Il ne doit généralement pas contenir `.venv/`, car cet environnement dépend de la machine qui l'a créé et peut être reconstruit.

13.4 Cas D — Changement d'ordinateur

Installer une version compatible de Python, cloner le projet, recréer `.venv`, activer, puis exécuter `python -m pip install -r requirements.txt`.

13.5 Cas E — `ModuleNotFoundError`

Erreur fréquente

`ModuleNotFoundError: No module named 'pandas'` signifie que l'interpréteur courant ne trouve pas `pandas` dans ses emplacements de modules. Cela ne prouve pas que `pandas` n'est installé nulle part sur la machine.

Procédure :

- ◆ afficher `sys.executable`;
- ◆ vérifier le `venv`;
- ◆ exécuter `python -m pip -version`;
- ◆ exécuter `python -m pip list`;
- ◆ vérifier l'interpréteur sélectionné dans VS Code;
- ◆ installer le paquet avec `python -m pip install pandas` si nécessaire.

13.6 Cas F — pip installe dans le mauvais Python

Si `pip -version` et `python -m pip -version` pointent vers des emplacements différents, utilisez la seconde forme et clarifiez le `PATH`.

Licence IRM

Tableau de dépannage

Symptôme	Cause probable	Vérification	Solution
<code>python</code> non reconnu	Python absent du PATH ou non installé	<code>where python / which python3</code>	Réparer l'installation ou utiliser le lanceur approprié; rouvrir le terminal.
<code>pip</code> non reconnu	Scripts non accessibles ou mauvais Python	<code>python -m pip -version</code>	Utiliser <code>python -m pip</code> ; vérifier l'installation de <code>pip</code> .
<code>ModuleNotFoundError</code>	Paquet absent dans l'interpréteur courant	<code>sys.executable, python -m pip list</code>	Installer le paquet dans le bon environnement.
Paquet installé mais introuvable	Installation faite dans un autre Python	Comparer chemins de <code>pip</code> et Python	Utiliser <code>python -m pip</code> ; sélectionner le bon interpréteur.
Import rouge dans VS Code	Mauvais interpréteur ou analyseur non synchronisé	Python: Select Interpreter	Sélectionner l'environnement correct et recharger si nécessaire.
Activation PowerShell refusée	Politique d'exécution du shell	Lire le message PowerShell	Utiliser une politique adaptée à l'organisation ou un autre shell autorisé; ne pas désactiver la sécurité sans comprendre.
Plusieurs Python	PATH ambigu	<code>where python, py -list</code>	Choisir explicitement la version et recréer le <code>venv</code> avec elle.
<code>conda</code> non reconnu	Initialisation du shell absente ou Conda non installé	<code>conda -version</code>	Ouvrir le shell approprié ou initialiser Conda selon la documentation officielle.
Terminal affiche (base)	Activation automatique de base	<code>conda info -envs</code>	Ce n'est pas une erreur; ajuster la configuration seulement si nécessaire.
<code>pip install</code> permissions	Installation globale protégée	Chemin de <code>sys.executable</code>	Créer un <code>venv</code> au lieu d'installer globalement.

Symptôme	Cause probable	Vérification	Solution
<code>requirements.txt</code> échoue ailleurs	Incompatibilité OS, Python ou binaire	Vérifier versions et messages d'erreur	Aligner Python, adapter les dépendances ou documenter les contraintes système.

Licence IRM

Checklist universelle de diagnostic

Avant toute réinstallation, exécuter :

Terminal

```
python --version
python -c "import sys; print(sys.executable)"
python -m pip --version
python -m pip list
```

Puis, dans VS Code :

```
import sys
print(sys.executable)
```

Question centrale : **est-ce bien le même Python ?**

PARTIE VIII

Méthode de référence et bonnes pratiques

Licence IRM

Construire un projet Python reproductible : la méthode de référence

16.1 Variante A — Python + venv + pip

1 — Créer le projet

Terminal

```
mkdir analyse_donnees  
cd analyse_donnees
```

2 — Créer l'environnement

Terminal

```
python -m venv .venv
```

3 — Activer et vérifier

Activer selon le système puis :

Terminal

```
python -c "import sys; print(sys.executable)"
```

4 — Installer les dépendances

Terminal

```
python -m pip install --upgrade pip  
python -m pip install numpy pandas matplotlib scikit-learn
```

5 — Figer

Terminal

```
python -m pip freeze > requirements.txt
```

6 — Versionner et partager

Versionner le code, `requirements.txt`, `README.md` et les configurations utiles. Exclure `.venv`.

7 — Reconstruire

Terminal

```
python -m venv .venv
# activer selon le système
python -m pip install -r requirements.txt
```

16.2 Variante B — Conda

Terminal

```
conda create -n analyse_donnees python=3.12
conda activate analyse_donnees
conda install numpy pandas matplotlib scikit-learn
conda env export > environment.yml
```

Sur une autre machine :

Terminal

```
conda env create -f environment.yml
conda activate analyse_donnees
```

Bonnes pratiques professionnelles

- ◆ Un projet = un environnement isolé.
- ◆ Éviter les installations globales inutiles.
- ◆ Préférer `python -m pip` lorsque l'on veut lever l'ambiguïté sur l'interpréteur.
- ◆ Ne pas versionner ni envoyer `.venv`.
- ◆ Versionner les fichiers décrivant l'environnement.
- ◆ Documenter la version de Python.
- ◆ Figurer les dépendances lorsque la reproductibilité l'exige.
- ◆ Tester réellement la reconstruction sur une autre machine ou dans un environnement propre.
- ◆ Ne pas supposer que VS Code et le terminal utilisent toujours le même Python.
- ◆ Vérifier `sys.executable` dès qu'un problème d'environnement est suspecté.
- ◆ Ne jamais modifier manuellement les fichiers internes d'un `venv` sans raison experte.
- ◆ Ne pas altérer aveuglément le Python système sous macOS/Linux.
- ◆ Pour les projets IA, documenter aussi les dépendances GPU et matérielles pertinentes.

Fiche mémo — Environnements Python

Vérifier Python

Terminal

```
python --version  
python -c "import sys; print(sys.executable)"
```

Créer un venv

Terminal

```
python -m venv .venv
```

Activer

WINDOWS POWERSHELL

Terminal

```
.\.venv\Scripts\Activate.ps1
```

WINDOWS CMD

Terminal

```
.\venv\Scripts\activate.bat
```

macOS / LINUX

Terminal

```
source .venv/bin/activate
```

Installer et inspecter

Terminal

```
python -m pip install package
python -m pip list
python -m pip show package
```

Figurer et reconstruire

Terminal

```
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Conda

Terminal

```
conda create -n mon_projet python=3.12
conda activate mon_projet
conda install numpy pandas
conda env export > environment.yml
conda env create -f environment.yml
```

Conclusion

Pour de nombreux projets Python, la chaîne :

Python + VS Code + venv + pip + requirements.txt

constitue une approche légère, transparente et suffisamment robuste, à condition d'être appliquée avec discipline.

Conda propose une approche plus globale :

Conda + environnement Conda + environment.yml

particulièrement utile lorsqu'il faut gérer facilement plusieurs versions de Python, des dépendances binaires ou un environnement scientifique plus complexe.

À retenir

Le meilleur environnement n'est pas celui qui possède le plus d'outils, mais celui qui permet de **développer, partager et reconstruire le projet avec le moins d'ambiguïtés possible.**

Références et ressources officielles

- ◆ Python Software Foundation — téléchargements Python : <https://www.python.org/downloads/>
- ◆ Documentation Python — environnements virtuels : <https://docs.python.org/3/library/venv.html>
- ◆ Python Packaging User Guide — pip et environnements virtuels : <https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/>
- ◆ Visual Studio Code — environnements Python : <https://code.visualstudio.com/docs/python/environments>
- ◆ Visual Studio Code — démarrer avec Python : <https://code.visualstudio.com/docs/python/python-tutorial>
- ◆ Conda — gestion des environnements : <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>
- ◆ Conda — export des environnements : <https://docs.conda.io/projects/conda/en/latest/commands/export.html>
- ◆ Anaconda — comparaison Anaconda Distribution / Miniconda : <https://www.anaconda.com/docs/getting-started/concepts/anaconda-or-miniconda>